

Lebanese American University
Department of Computer Science and Mathematics

CSC 430 Computer Networks
Spring 2025-2026

DESIGN PROJECT

1. Description

In this project, you are required to create a caching proxy server that forwards requests from a client to a target server and relays back the response. It could include features like caching, blocking certain URLs, and logging requests.

2. Requirements

Write a python code to implement the following functions in your proxy server:

A. Basic Proxy Functionality:

- Accept requests from clients and forward these requests to a specified target server.
- Retrieve responses from the target server and relay them back to the client.
- Support both HTTP and as a BONUS HTTPS requests, with HTTPS being forwarded securely without decrypting.

B. Socket Programming:

- Use sockets to manage connections between the client and the target server.
- Set up the server to listen for incoming client requests on a designated port.
- Establish a connection to the target server and facilitate data transfer between the two.

C. Request Parsing:

- Parse client requests to extract information such as the target server's address, port, and request type (GET, POST, etc.).
- Modify headers if necessary to comply with proxy forwarding requirements (e.g., setting the "Host" header or stripping other client-specific headers).

D. Threading: Handle multiple client requests concurrently using multithreading. This allows the proxy server to manage multiple connections simultaneously.

E. Logging: Implement logging to record details of requests and responses, such as: Client IP address and port; Target server address and port; HTTP method and URL requested; Timestamp of request and response; and Log error messages if requests fail.

F. Content Caching:

- Cache responses from the target server so repeated requests for the same resource can be served from the cache, reducing server load.
- Implement cache invalidation rules based on response headers (e.g., Cache-Control, Expires) or a custom cache timeout.

G. Blacklist/Whitelist:

- Add URL or IP filtering by maintaining a list of allowed (whitelisted) or disallowed (blacklisted) domains or IP addresses.
- Reject requests to blacklisted resources with a custom response.

H. HTTPS Proxy (BONUS):

- If you want to intercept and inspect HTTPS traffic (rather than just forward it), implement SSL/TLS decryption using an SSL certificate.
- This requires setting up a man-in-the-middle (MITM) structure with a self-signed certificate, which allows the proxy to decrypt HTTPS traffic.
- Be cautious with this as it impacts user privacy and security—this feature is generally used for debugging or educational purposes.

I. Admin Interface (BONUS):

- Build a web-based admin interface for viewing and managing logs, cache entries, and blacklist/whitelist configurations.
- Provide basic stats on proxy usage, cache performance, and active connections.

3. Grading

In this project you will work in the assigned groups and the grading will be based on the code, the report and the live demo.

4. Deliverables

At the end of this project, you should submit your (thoroughly documented) code along with a project report. In this report, you should describe your high-level approach, the challenges you faced, a list of properties/features of your design, and an overview of how you tested your code. Your report should explain each of the required functions. You should also include screenshots of your running application and testing.

- The submission of the Final Report with the full application and a readme file is due on **Sunday April 26th [End of Day] on Blackboard.**
- Project Demos will be held in the **Last week of the semester [Week of April. 27th].**

Important Notes:

- The bonus parts are worth an additional 10% on the total project grade.
- The grade includes the implementation decisions you have taken and the testing you have performed and how you explain these in your report.
- Each member of the group will be graded individually based on her/his input to the project.
- Code comments **MUST** clearly indicate which team member contributed which part of the code. Comments should also clarify the code.
- Code used from other sources (if any) must be clearly documented and flagged in the code by a comment.